Software Architecture and Design Specification

Project: Student Attendance Management System

Version: 1.0

Authors: Team 14

Date: 04-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 14	Initial SAD: architecture, design, API, security and UX for SAMS

Approvals

Role	Name	Signature/Date
QA Lead	Joel Williams	Joel 04-10-2026
Dev Lead	Jaladi Suhas	Suhas 04-10-2026
Product Owner	Kashyap VP	Kashyap 04-10-2026
Test Engineer	Krish Gupta	Krish 04-10-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Student Attendance Management System (SAMS). It translates the requirements defined in the SAMS SRS v1.0 into an architecture and design structure that can be built, tested, secured, and maintained. It also defines the major SAMS components, their responsibilities, interactions, technology choices, security considerations, and design decisions.

1.2 Scope

Covers the architecture and design of SAMS, including authentication and role-based access control (RBAC), user and course administration, session and attendance recording, attendance calculation, reporting and export, notifications, and audit. The document also covers the relationships between these components and the database, along with API design, security architecture, error handling, logging, monitoring, and UX considerations.

1.3 Audience

Developers, QA engineers, security reviewers, instructors and evaluators, and maintenance teams. Developers use the document to understand component boundaries and interfaces; QA engineers use it to derive and verify tests; security reviewers use it to review risks, threats, and logging rules; instructors and evaluators use it to assess design decisions and requirements coverage; and maintenance teams use it to understand the system structure and known issues.

1.4 Definitions

SAMS – Student Attendance Management System; SRS – Software Requirements Specification; RTM – Requirements Traceability Matrix; STP – Software Test Plan; RBAC – Role-Based Access Control; STRIDE – security threat-modeling approach covering common security threats; SAMS-F-### – functional requirement identifier; SAMS-NF-### – non-functional requirement identifier; SAMS-SR-### – security requirement identifier.

2. Document Overview

2.1 How to use this document

This document specifies the architecture and design of the Student Attendance Management System (SAMS). It turns the requirements in the SAMS SRS v1.0 into a structure that can be built, tested, and maintained. It provides the following architectural deliverables:

- UML component diagram and component descriptions (3.3, 3.4): the major building blocks of SAMS (authentication and RBAC, user and course administration, session and attendance recording, attendance calculation, reporting and export, notification, audit) and how they depend on each other and on the database.

- Architecture pattern and rationale (3.5): the chosen pattern and why the alternatives were rejected.

- Technology stack and data stores (3.6): the framework (Django/Flask or Spring Boot) and the relational database, as allowed by the SRS constraints.

- Security architecture and threat model (3.9): a STRIDE-based analysis tied to the security objectives SO-1 and SO-2 and to requirements SAMS-SR-001 to SAMS-SR-006.

- UML sequence diagrams (4.2): at least two SAMS-specific flows, for example a faculty member marking attendance and a faculty member correcting a saved record with an audit entry.

- API design (4.3): interface definitions for at least two components, including endpoints, request and response formats, and error codes.

- Error handling, logging, monitoring, and UX design (4.4, 4.5): how failures are reported safely, what is logged and what must never be logged, and how the interface meets the usability requirement SAMS-NF-004.

Different readers should start in different places:

Reader	Start with	Purpose
Developers	Sections 3.3–3.6, 4.2, 4.3	Understand component boundaries and interfaces before implementing
QA engineers	Sections 3.8, 4.3, 4.4	Derive test cases and check them against the design
Security reviewers	Sections 3.7, 3.9, 4.4	Review risks, the threat model, and logging rules
Instructors and evaluators	Sections 1, 3.5, 3.8	Check design decisions and requirements coverage
Maintenance teams	Sections 3.4, 3.6, 4.6	Understand the structure and known open issues

Section 3 describes the architecture (what the system is made of and why). Section 4 describes the design (how the components behave and interact). The appendices hold the glossary, references, and tools.

2.2 Related Documents

Document	Version / Status	Relationship to this document
Software Requirements Specification (SRS)	v1.0, 05-09-2026	Source of all functional (SAMS-F-###), non-functional (SAMS-NF-###), and security (SAMS-SR-###) requirements that this design must satisfy
Requirements Traceability Matrix (RTM)	SRS Section 8	Maps each requirement to a design specification and module; this document defines those modules (see Section 3.8)
Software Test Plan (STP)	v1.0, draft	Uses this design to plan integration, system, and security testing; its test environment and tools follow the stack chosen in Section 3.6
IEEE 42010, OWASP guidelines, NIST SP 800-160	Referenced standards	Guide the architecture description, security design, and threat modelling approach (listed in Appendix 5.2)

3. Architecture

3.1 Goals & Constraints

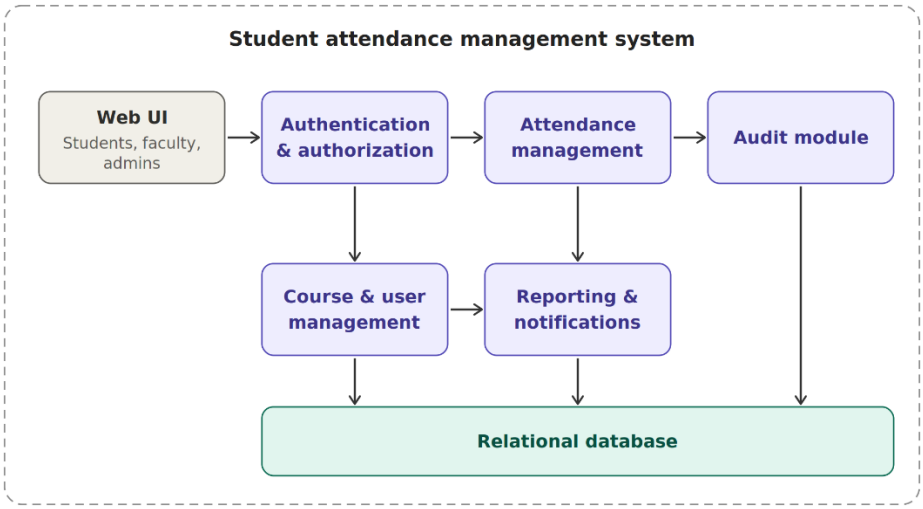
Goals: accurate attendance recording, secure student data, reliable reporting, role-based access, and maintainability. Constraints: institutional privacy requirements, web-based operation, database availability, and the selected Python (Django/Flask) or Java (Spring Boot) technology stack.

3.2 Stakeholders & Concerns

Students: privacy and easy access to attendance information.
Faculty: accurate and efficient attendance marking and reporting.
Administrators: user/course management, reporting, and system control.
Developers: modularity and maintainability.

3.3 Component (UML) Diagram

The component diagram shows the Web UI connected to Authentication & Authorization, Attendance Management, Course & User Management, Reporting & Notifications, Audit, and the relational Database.



3.4 Component Descriptions

- Web UI: provides role-specific interfaces for students, faculty, and administrators.
- Authentication & Authorization: authenticates users and enforces role-based access.
- Attendance Management: creates attendance sessions, records attendance, prevents duplicate records, and handles attendance corrections.
- Course & User Management: manages users, courses, class sections, and enrollments.
- Reporting & Notifications: generates attendance reports and handles low-attendance notifications.
- Audit Module: records attendance changes and administrative/security actions.
- Relational Database: stores user, course, attendance, correction, report, and audit data.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is chosen for clear separation of concerns. The presentation layer handles user interaction, the application/service layer implements attendance and business rules, the data-access layer handles persistence, and the database stores system data. This structure is suitable for the Student Attendance Management System because it is straightforward to develop, test, and maintain.

3.6 Technology Stack & Data Stores

Python (Django/Flask) or Java (Spring Boot) for the backend, web technologies for the user interface, and MySQL or PostgreSQL as the relational database. HTTPS/TLS is used for secure communication.

3.7 Risks & Mitigations

Risk: incorrect or duplicate attendance records -> Mitigation: input validation, database constraints, and transaction controls.
Risk: unauthorized access to student data -> Mitigation: authentication, RBAC, and secure communication.
Risk: service/database failure -> Mitigation: backups, error handling, and recovery procedures.

3.8 Traceability to Requirements

Attendance marking requirements -> Attendance Management.
User authentication and authorization requirements -> Authentication & Authorization.
Course and user management requirements -> Course & User Management.
Attendance reporting and notification requirements -> Reporting & Notifications.
Audit requirements -> Audit Module.

3.9 Security Architecture

Security is based on authentication, role-based access control, HTTPS/TLS, secure password hashing, protection of sensitive attendance data, and audit logging. STRIDE considerations include: Spoofing -> authentication; Tampering -> input validation and database controls; Information Disclosure -> TLS and access control; Denial of Service -> rate limiting and monitoring; Elevation of Privilege -> RBAC.

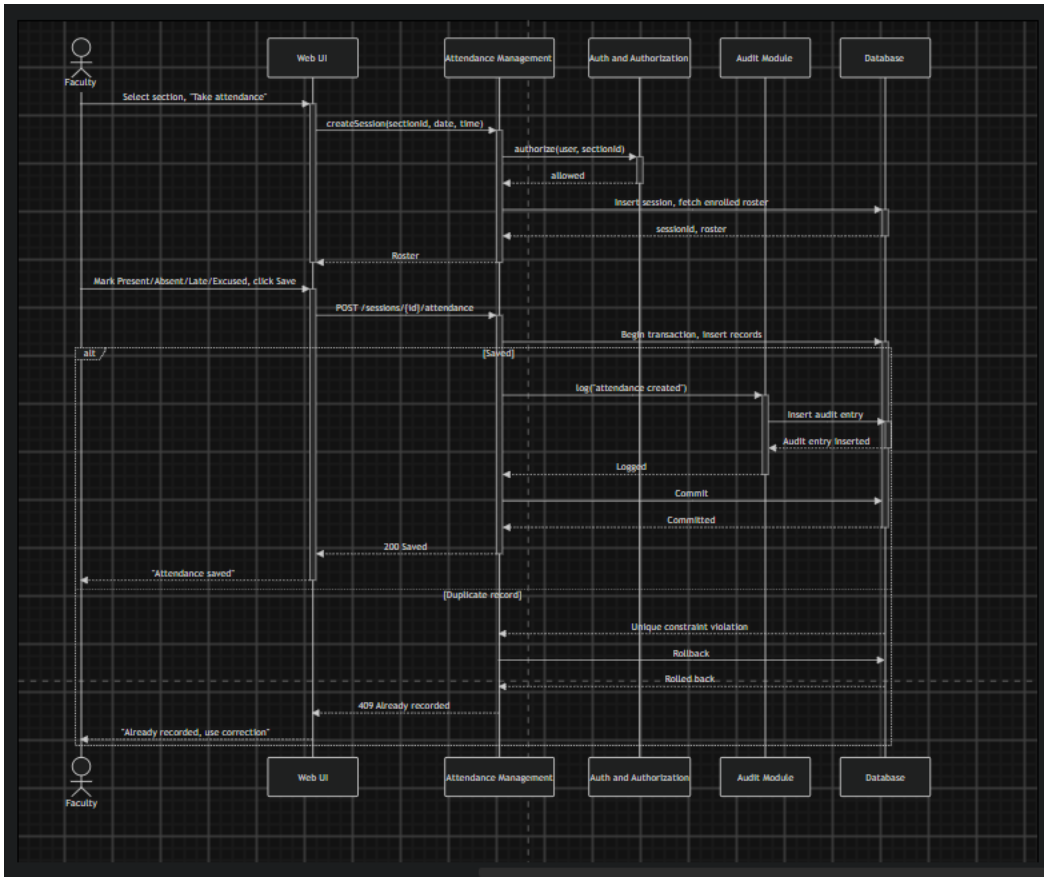
4. Design

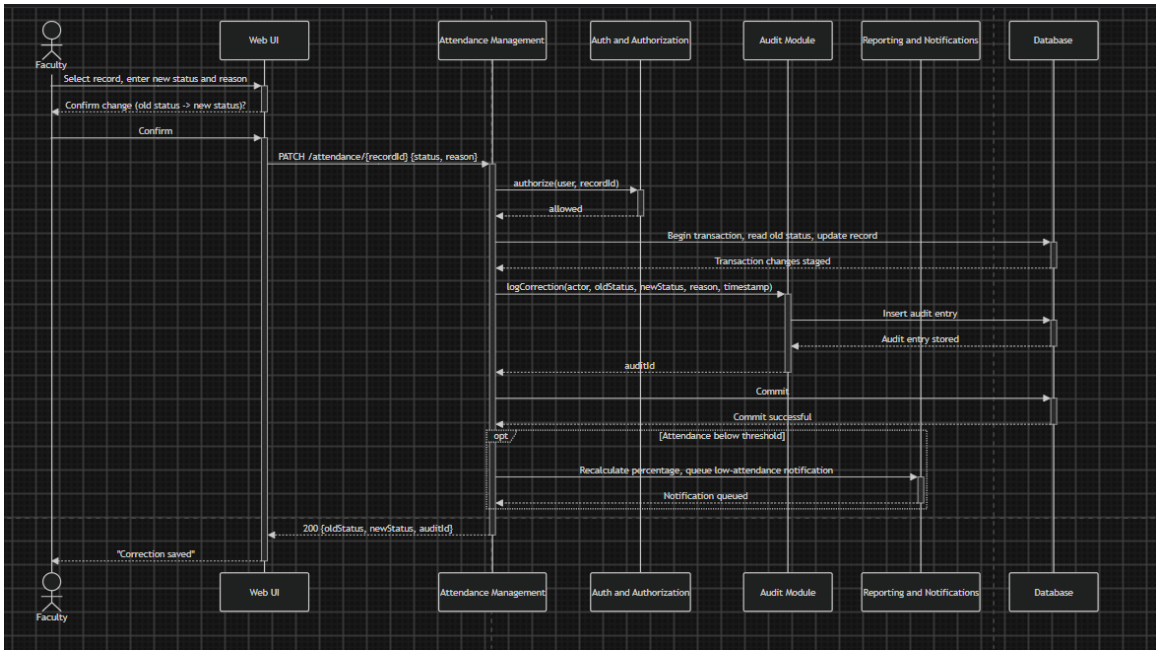
4.1 Design Overview

SAMS follows the layered architecture in Section 3.5: Web UI -> RBAC check -> service layer -> data-access layer -> database. Key decisions:

- Every page and API endpoint is authorized on the server by role and section assignment (SAMS-SR-003).
- Attendance is saved in one transaction with a unique (student, session) constraint to prevent duplicates (SAMS-F-009, SAMS-NF-006).
- A correction and its audit entry are saved together in the same transaction (SAMS-F-010, SAMS-SR-005).
- Attendance percentages are calculated from stored records; the threshold is a configurable setting (SAMS-F-011, SAMS-F-014).

4.2 UML Sequence Diagrams





4.3 API Design

REST/JSON over HTTPS. Errors return {code, message} with standard HTTP status codes.

Authentication & Authorization
Endpoint: /api/v1/auth/login
Method: POST
Request: {username, password}
Response: {status, role} + secure session cookie
Errors: 401 Invalid credentials, 429 Too many failed attempts.

Attendance Management
Endpoint: /api/v1/sessions/{sessionId}/attendance
Method: POST
Request: {records: [{studentId, status}]}
Response: {status, savedCount}
Errors: 400 Invalid status or student not enrolled, 403 Not assigned to section, 409 Attendance already recorded.

Reporting & Notifications
Endpoint: /api/v1/reports/shortage?sectionId={id}
Method: GET
Request: sectionId, optional threshold
Response: {threshold, students: [{studentId, percentage}]}
Errors: 400 Invalid filter, 403 Section not authorized.

4.4 Error Handling, Logging & Monitoring

Standardized, user-friendly error messages with no stack traces or server details. Failed attendance saves are rolled back completely. Audit logs record actor, timestamp, action, target, and outcome for logins, attendance changes, user management, threshold changes, and exports; passwords and session tokens are never logged. Monitoring covers response time (SAMS-NF-001), uptime (SAMS-NF-002), error rate, failed logins, and daily backup status.

4.5 UX Design

Role-specific dashboards, a roster-based attendance screen with Present/Absent/Late/Excused controls, confirmation prompts for corrections, and report filters with CSV/PDF export. Responsive layout, keyboard navigation, and clear form labels (SAMS-NF-004).

4.6 Open Issues & Next Steps

Open issues: final framework choice, how Late/Excused count towards the percentage, and login-lockout settings.
Future enhancements: QR/RFID attendance, ERP integration, and a mobile app.

5. Appendices

5.1 Glossary: SAMS, RBAC, STRIDE, TLS, CSRF, API, ERP, Attendance Session, Roster, Attendance Threshold.
5.2 References: SAMS SRS v1.0, SAMS STP v1.0, ISO/IEC/IEEE 42010, OWASP Top 10, NIST SP 800-160, WCAG 2.1.
5.3 Tools: PlantUML, draw.io.